

Plan de Desarrollo Completo para CRM Legal

Resumen Ejecutivo

Este documento presenta un plan de desarrollo integral para completar el CRM Legal, un sistema de gestión para bufetes y profesionales legales desarrollado en Python con arquitectura MVC, interfaz tkinter y base de datos SQLite. El plan se basa en el análisis técnico del sistema actual, que muestra una base sólida con funcionalidades core ya implementadas y dos branches de desarrollo pendientes de integrarse.

El plan propuesto estructura el desarrollo en seis fases priorizadas estratégicamente, con un horizonte temporal estimado de 7-8 meses. La primera fase está enfocada en la consolidación del código existente, mientras que las fases posteriores implementan nuevas funcionalidades críticas, mejoras de UX, capacidades avanzadas de analytics y funcionalidades de IA.

Equipo Recomendado: 1-2 desarrolladores Python con experiencia en aplicaciones de escritorio y bases de datos.

Timeline General: 31-37 semanas (7-8 meses) para completar todas las fases.

Prioridades Críticas: Integración inmediata de branches pendientes, sistema de etiquetas global, módulo financiero, mejoras de búsqueda y usabilidad.

Índice

1. [Hoja de Ruta Priorizada](#)
2. [Especificaciones Técnicas por Módulo](#)
3. [Arquitectura de Integración](#)
4. [Timeline Detallado](#)

- 5. [Recursos Necesarios](#)
 - 6. [Guías de Implementación](#)
 - 7. [Plan de Testing](#)
 - 8. [Consideraciones de Mantenimiento](#)
 - 9. [Resumen por Fase](#)
-

1. Hoja de Ruta Priorizada

Fase 1: Consolidación y Fortalecimiento (3 semanas)

- **Objetivo:** Integrar funcionalidades existentes y establecer bases para desarrollo futuro
- **Prioridad:** Crítica - Precede a todas las demás fases
- **Tareas Principales:**
 1. Merge de feature/modulo-tareas
 2. Merge de feature/gestion-partes
 3. Implementación de test suite básico
 4. Documentación técnica completa
 5. Refactorización de áreas críticas
 6. Corrección de bugs identificados

Fase 2: Funcionalidades Core (6 semanas)

- **Objetivo:** Implementar módulos críticos que complementan la funcionalidad base
- **Prioridad:** Alta - Funcionalidades de alto impacto para usuarios
- **Tareas Principales:**
 1. Gestor de Etiquetas Global
 2. Módulo de Presupuestos/Honorarios
 3. Sistema de Búsqueda Global
 4. Mejoras críticas de UI/UX

Fase 3: Mejoras de Usabilidad (4 semanas)

- **Objetivo:** Optimizar la experiencia de usuario
- **Prioridad:** Media-Alta - Impacta directamente en la eficiencia de uso
- **Tareas Principales:**
 1. Navegación mejorada (shortcuts, breadcrumbs)
 2. Gestión avanzada de archivos
 3. Personalización de interfaz
 4. Optimizaciones de performance

Fase 4: Analytics y Reportes (5 semanas)

- **Objetivo:** Proporcionar insights y capacidades de reporting
- **Prioridad:** Media - Valor agregado importante
- **Tareas Principales:**
 1. Dashboard principal con KPIs
 2. Sistema de reportes configurable
 3. Gráficos de análisis y tendencias
 4. Exportación avanzada (PDF, Excel)

Fase 5: Integración de IA (8 semanas)

- **Objetivo:** Añadir capacidades inteligentes al sistema
- **Prioridad:** Media-Baja - Valor diferencial pero complejidad alta
- **Tareas Principales:**
 1. Análisis automático de documentos
 2. Generación de contenido legal
 3. Asistente virtual integrado
 4. Analytics predictivos

Fase 6: Escalabilidad y Proyección (5 semanas)

- **Objetivo:** Preparar el sistema para crecimiento futuro
- **Prioridad:** Baja - Valor a largo plazo

- **Tareas Principales:**

1. Arquitectura multi-usuario
 2. API REST para integraciones
 3. Integración con servicios cloud
 4. Preparación para companion mobile
-

2. Especificaciones Técnicas por Módulo

2.1 Gestor de Etiquetas Global

Descripción: Sistema transversal de categorización que permite etiquetar y filtrar entidades a través de todo el CRM.

Especificaciones técnicas:

Cambios en Base de Datos

```
-- Nueva tabla de etiquetas
CREATE TABLE etiquetas (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    color TEXT NOT NULL DEFAULT '#3498db',
    descripcion TEXT,
    created_at INTEGER,
    UNIQUE(nombre)
);

-- Tabla de relación para vincular etiquetas con diferentes
entidades
CREATE TABLE etiquetas_relaciones (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    etiqueta_id INTEGER NOT NULL,
    entidad_tipo TEXT NOT NULL, -- 'cliente', 'caso', 'actividad',
'documento', etc.
    entidad_id INTEGER NOT NULL,
    created_at INTEGER,
    FOREIGN KEY (etiqueta_id) REFERENCES etiquetas(id) ON DELETE
CASCADE,
    UNIQUE(etiqueta_id, entidad_tipo, entidad_id)
);

-- Índices para búsquedas eficientes
CREATE INDEX idx_etiquetas_nombre ON etiquetas(nombre);
CREATE INDEX idx_etiquetas_rel_tipo_id ON
etiquetas_relaciones(entidad_tipo, entidad_id);
CREATE INDEX idx_etiquetas_rel_etiqueta ON
etiquetas_relaciones(etiqueta_id);
```

Funciones de Capa de Datos (crm_database.py)

- `add_etiqueta(nombre, color, descripcion)`: Crea una nueva etiqueta
- `get_etiquetas()`: Obtiene todas las etiquetas disponibles
- `get_etiquetas_by_entidad(entidad_tipo, entidad_id)`: Obtiene etiquetas de una entidad específica
- `asignar_etiqueta(etiqueta_id, entidad_tipo, entidad_id)`: Asigna etiqueta a entidad
- `desasignar_etiqueta(etiqueta_id, entidad_tipo, entidad_id)`: Remueve asignación
- `actualizar_etiqueta(etiqueta_id, nombre, color, descripcion)`: Modifica etiqueta
- `eliminar_etiqueta(etiqueta_id)`: Elimina etiqueta y todas sus relaciones

Módulo de UI (etiquetas_ui.py)

- `EtiquetasManager`: Clase para gestionar etiquetas (CRUD completo)
- `EtiquetasSelectorDialog`: Diálogo para asignar/desasignar etiquetas
- `EtiquetasFilterBar`: Barra de filtrado por etiquetas
- Integración visual: Badges de colores para mostrar etiquetas en TreeViews

Integración con Módulos Existentes

- Modificar todas las vistas de lista para mostrar etiquetas
- Añadir filtrado por etiquetas en todas las vistas relevantes
- Añadir botón "Gestionar Etiquetas" en cada vista principal
- Implementar búsqueda por etiquetas

2.2 Módulo de Presupuestos/Honorarios

Descripción: Sistema completo de gestión financiera para presupuestos, honorarios, pagos y facturación.

Especificaciones técnicas:

Cambios en Base de Datos


```

-- Tabla de presupuestos
CREATE TABLE presupuestos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    caso_id INTEGER NOT NULL,
    titulo TEXT NOT NULL,
    descripcion TEXT,
    monto_total REAL NOT NULL,
    moneda TEXT NOT NULL DEFAULT 'EUR',
    estado TEXT NOT NULL DEFAULT 'borrador', -- borrador, enviado,
aceptado, rechazado
    fecha_emision TEXT, -- YYYY-MM-DD
    fecha_aceptacion TEXT, -- YYYY-MM-DD
    notas TEXT,
    created_at INTEGER,
    updated_at INTEGER,
    FOREIGN KEY (caso_id) REFERENCES casos(id) ON DELETE CASCADE
);

-- Tabla de conceptos/ítems del presupuesto
CREATE TABLE presupuesto_items (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    presupuesto_id INTEGER NOT NULL,
    descripcion TEXT NOT NULL,
    cantidad REAL NOT NULL DEFAULT 1,
    precio_unitario REAL NOT NULL,
    subtotal REAL NOT NULL,
    notas TEXT,
    orden INTEGER NOT NULL DEFAULT 1,
    FOREIGN KEY (presupuesto_id) REFERENCES presupuestos(id) ON
DELETE CASCADE
);

-- Tabla de pagos recibidos
CREATE TABLE pagos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

presupuesto_id INTEGER,
caso_id INTEGER NOT NULL,
monto REAL NOT NULL,
moneda TEXT NOT NULL DEFAULT 'EUR',
fecha TEXT NOT NULL, -- YYYY-MM-DD
metodo TEXT NOT NULL, -- efectivo, transferencia, etc
referencia TEXT, -- número de factura, etc
notas TEXT,
created_at INTEGER,
FOREIGN KEY (presupuesto_id) REFERENCES presupuestos(id) ON
DELETE SET NULL,
FOREIGN KEY (caso_id) REFERENCES casos(id) ON DELETE CASCADE
);

-- Índices para consultas frecuentes
CREATE INDEX idx_presupuestos_caso ON presupuestos(caso_id);
CREATE INDEX idx_presupuestos_estado ON presupuestos(estado);
CREATE INDEX idx_pagos_caso ON pagos(caso_id);
CREATE INDEX idx_pagos_fecha ON pagos(fecha);

```

Funciones de Capa de Datos

- `add_presupuesto(caso_id, titulo, descripcion, monto_total, moneda, estado, fecha_emision, notas)`: Crear presupuesto
- `add_presupuesto_item(presupuesto_id, descripcion, cantidad, precio_unitario, subtotal, notas, orden)`: Añadir ítem
- `update_presupuesto_estado(presupuesto_id, nuevo_estado, fecha_aceptacion=None)`: Cambiar estado
- `get_presupuestos_by_caso(caso_id)`: Listar presupuestos de un caso
- `get_presupuesto_items(presupuesto_id)`: Obtener ítems de un presupuesto
- `add_pago(caso_id, monto, moneda, fecha, metodo, referencia, notas, presupuesto_id=None)`: Registrar pago
- `get_pagos_by_caso(caso_id)`: Listar pagos de un caso

- `get_balance_caso(caso_id)` : Calcular balance financiero del caso

Módulo de UI (finanzas_ui.py)

- `PresupuestosTab` : Pestaña para gestión de presupuestos
- `PresupuestoDialog` : Diálogo para crear/editar presupuestos
- `PresupuestoDetailView` : Vista detallada de presupuesto
- `PagosTab` : Pestaña para gestión de pagos
- `ReporteFinancieroView` : Vista de reportes financieros

Características de Exportación

- Generación de presupuestos en PDF
- Generación de facturas básicas
- Exportación de reportes financieros a Excel

2.3 Sistema de Búsqueda Global

Descripción: Motor de búsqueda unificado que permite buscar a través de todas las entidades del sistema.

Especificaciones técnicas:

Cambios en Base de Datos

```
-- Uso de SQLite FTS5 para búsqueda de texto completo
CREATE VIRTUAL TABLE busqueda_global USING fts5(
    content='',
    entidad_tipo,
    entidad_id,
    contenido
);

-- Trigger para mantener actualizado el índice de clientes
CREATE TRIGGER actualizar_busqueda_cliente AFTER UPDATE ON clientes
BEGIN
    DELETE FROM busqueda_global WHERE entidad_tipo = 'cliente' AND
entidad_id = old.id;
    INSERT INTO busqueda_global(entidad_tipo, entidad_id,
contenido)
    VALUES ('cliente', new.id, new.nombre || ' ' || new.direccion
|| ' ' || new.email || ' ' || new.whatsapp);
END;

-- Triggers similares para casos, audiencias, actividades, etc.
```

Funciones de Capa de Datos

- `rebuild_search_index()`: Reconstruye el índice de búsqueda completo
- `update_search_index(entidad_tipo, entidad_id)`: Actualiza una entidad específica
- `search_global(query, filters=None)`: Búsqueda con filtros opcionales
- `search_by_entity(query, entidad_tipo)`: Búsqueda en un tipo específico

Módulo de UI (busqueda_ui.py)

- `BusquedaGlobalDialog`: Diálogo principal de búsqueda

- `ResultadosBusquedaView`: Vista de resultados categorizada
- `SearchBar`: Componente de búsqueda rápida para barra principal

Características Avanzadas

- Autocompletado de términos
- Filtros por tipo de entidad, fecha, etiquetas
- Resaltado de términos en resultados
- Navegación directa a entidades desde resultados

2.4 Dashboard e Informes

Descripción: Sistema de analytics y reportes que provee métricas e insights sobre la actividad del bufete.

Especificaciones técnicas:

Dependencias Adicionales

```
matplotlib==3.7.1
reportlab==3.6.12
pandas==1.5.3
openpyxl==3.1.2
```

Módulo de UI (`analytics_ui.py`)

- `DashboardTab`: Pestaña principal con widgets de KPIs
- `GraficosView`: Visualizaciones y gráficos interactivos
- `ReportesManager`: Generador de reportes personalizados
- `KPIWidget`: Componente reutilizable para métricas

Tipos de Gráficos

- Barras: Casos por estado, audiencias por mes

- Circular: Distribución de tipos de casos
- Líneas: Evolución temporal de actividad
- Gantt: Cronograma de casos

Reportes Personalizables

- Actividad por período
- Estado de casos
- Rendimiento financiero
- Cargas de trabajo
- Calendarios y programación

Exportación

- PDF: Reportes formateados profesionalmente
- Excel: Datos tabulares para análisis adicional
- CSV: Exportación de datos crudos

2.5 Integración de IA

Descripción: Capacidades de inteligencia artificial para análisis de documentos, generación de contenido y asistencia.

Especificaciones técnicas:

Dependencias Adicionales

```
langchain==0.0.267  
sentence-transformers==2.2.2  
python-docx==0.8.11  
pypdf2==3.0.1  
pytesseract==0.3.10
```

Módulo de Integración IA (ia_engine.py)

- `DocumentProcessor`: Extracción de texto y metadatos
- `ContentGenerator`: Generación de documentos y textos
- `LegalAssistant`: Asistente virtual para consultas
- `PredictiveAnalytics`: Análisis predictivo de casos

Características de Procesamiento de Documentos

- OCR para documentos escaneados
- Extracción de entidades nombradas (personas, lugares, fechas)
- Clasificación automática de documentos
- Generación de resúmenes

Generación de Contenido

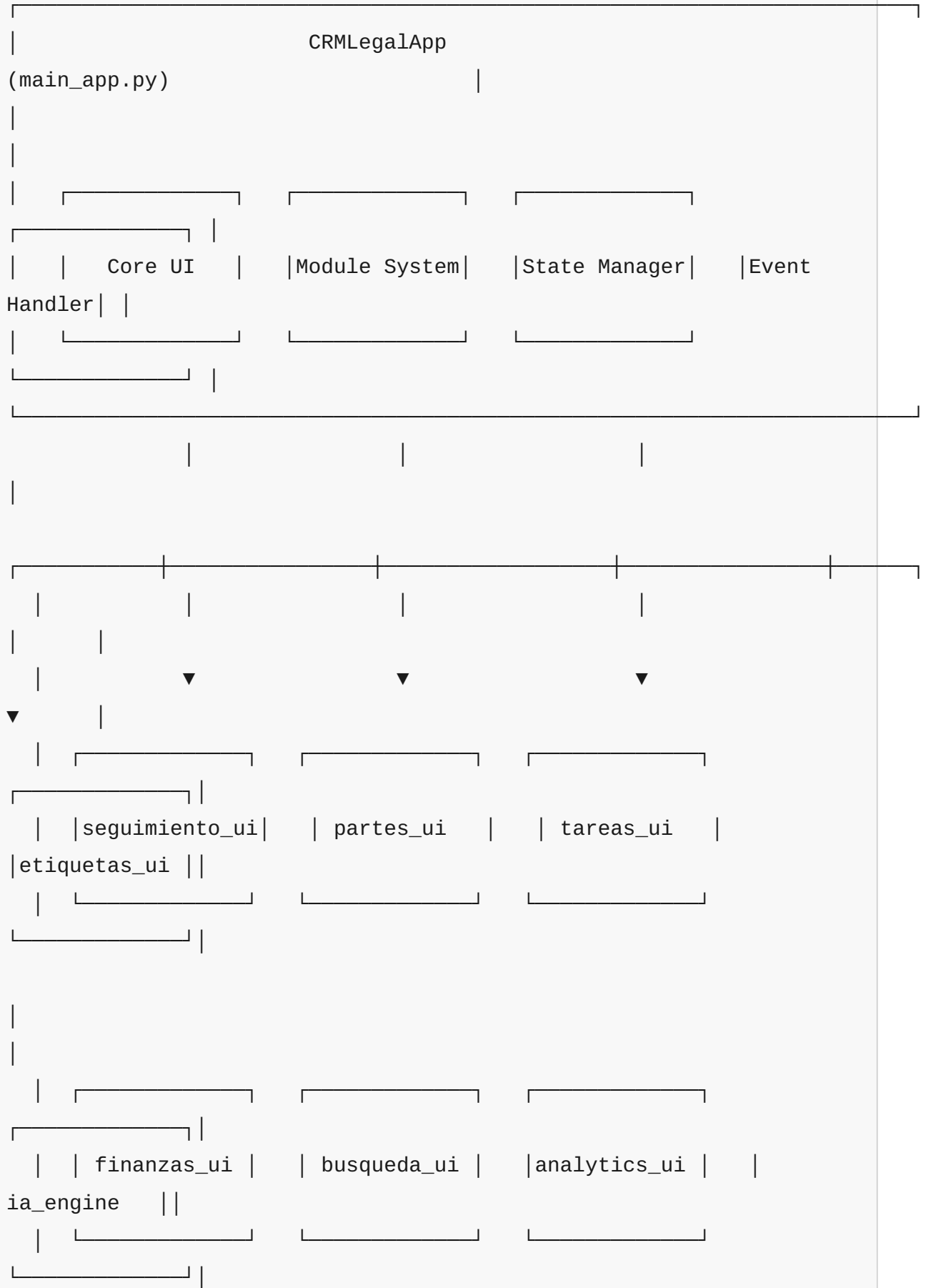
- Templates de documentos legales
- Autocompletado de cláusulas
- Sugerencias contextuales
- Redacción asistida

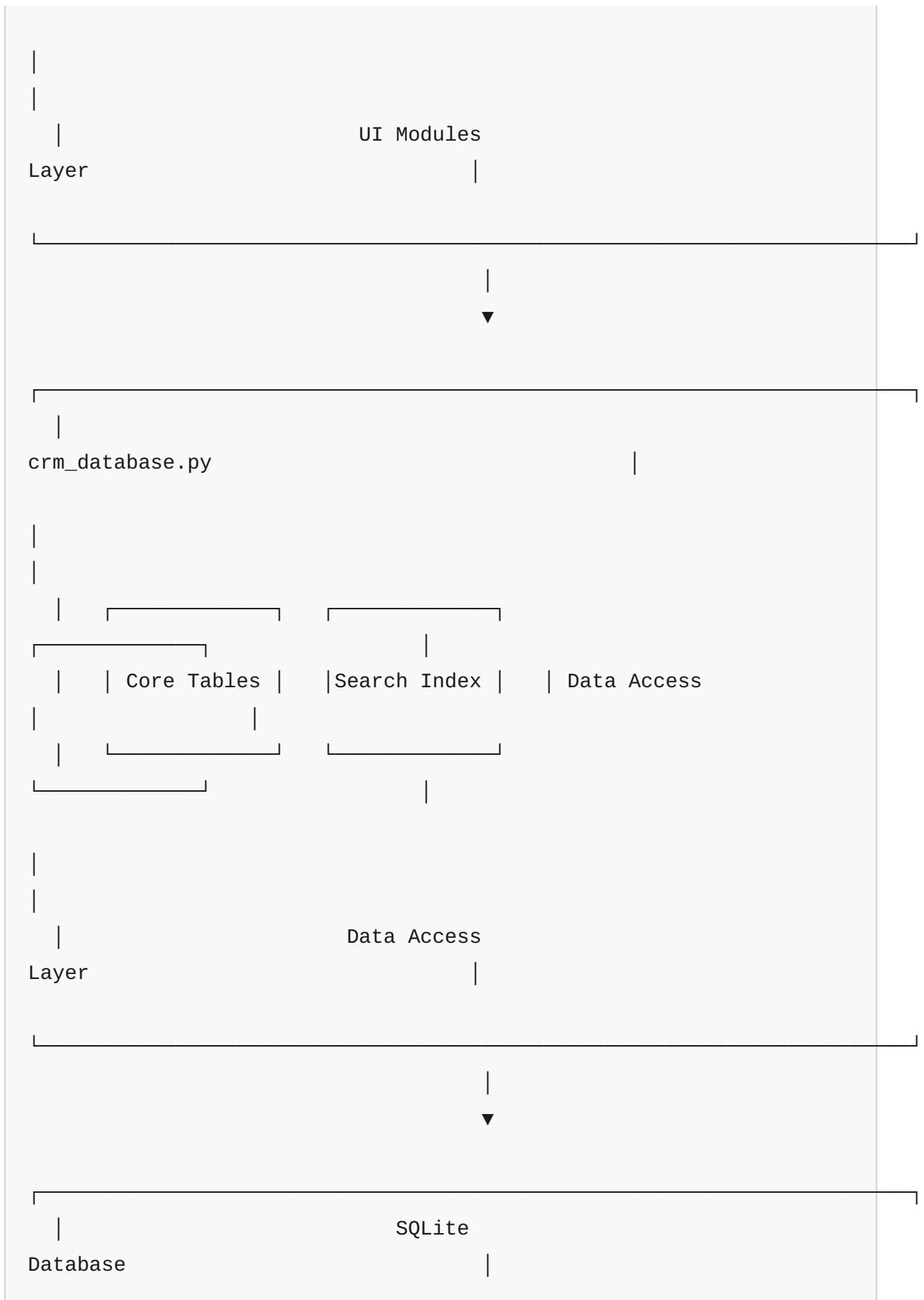
Asistente Virtual

- Interfaz de chat integrada
 - Consultas en lenguaje natural
 - Respuestas basadas en base de conocimiento
 - Recordatorios inteligentes
-

3. Arquitectura de Integración

3.1 Diagrama de Arquitectura Global





3.2 Estrategia de Integración de Módulos

Sistema de Registro de Módulos

```
class ModuleRegistry:
    def __init__(self):
        self.modules = {}
        self.tab_map = {}

    def register_module(self, module_name, module_class,
tab_name=None):
        self.modules[module_name] = module_class
        if tab_name:
            self.tab_map[module_name] = tab_name

    def get_module(self, module_name):
        return self.modules.get(module_name)

    def initialize_modules(self, parent, app_controller):
        initialized_modules = {}
        for name, module_class in self.modules.items():
            initialized_modules[name] = module_class(parent,
app_controller)
        return initialized_modules
```

Integración de Pestañas en el Sistema Existente

```
# En main_app.py
def create_widgets(self):
    # Código existente...

    # Inicializar registro de módulos
    self.module_registry = ModuleRegistry()

    # Registrar módulos
    self.module_registry.register_module('seguimiento',
SeguimientoTab, 'Seguimiento')
    self.module_registry.register_module('partes', PartesTab,
'Partes')
    self.module_registry.register_module('tareas', TareasTab,
'Tareas')
    self.module_registry.register_module('etiquetas', EtiquetasTab,
'Etiquetas')
    self.module_registry.register_module('finanzas', FinanzasTab,
'Finanzas')
    self.module_registry.register_module('analytics', DashboardTab,
'Dashboard')

    # Inicializar módulos y crear pestañas
    self.modules = {}
    for name, module in
self.module_registry.initialize_modules(self.main_notebook,
self).items():
        self.modules[name] = module
        tab_name = self.module_registry.tab_map.get(name)
        if tab_name:
            self.main_notebook.add(module, text=tab_name)
```

3.3 Sistema de Eventos para Comunicación entre Módulos

```
class EventBus:
    def __init__(self):
        self.subscribers = {}

    def subscribe(self, event_type, callback):
        if event_type not in self.subscribers:
            self.subscribers[event_type] = []
        self.subscribers[event_type].append(callback)

    def publish(self, event_type, data=None):
        if event_type in self.subscribers:
            for callback in self.subscribers[event_type]:
                callback(data)

# Uso en main_app.py
self.event_bus = EventBus()
self.event_bus.subscribe('caso_seleccionado',
self.modules['finanzas'].on_caso_selected)
self.event_bus.subscribe('etiqueta_creada',
self.update_etiquetas_ui)

# Publicar evento
def on_case_select(self, event):
    # Código existente...
    self.event_bus.publish('caso_seleccionado', self.selected_case)
```

3.4 Sistema de Menú Unificado

```

def crear_menu_principal(self):
    menubar = tk.Menu(self.root)

    # Menú Archivo
    file_menu = tk.Menu(menubar, tearoff=0)
    file_menu.add_command(label="Mostrar Ventana",
command=self._mostrar_ventana_callback)
    file_menu.add_separator()
    file_menu.add_command(label="Ocultar a Bandeja",
command=self.ocultar_a_bandeja)
    file_menu.add_separator()
    file_menu.add_command(label="Salir",
command=self.cerrar_aplicacion_directamente)
    menubar.add_cascade(label="Archivo", menu=file_menu)

    # Menú Buscar
    search_menu = tk.Menu(menubar, tearoff=0)
    search_menu.add_command(label="Búsqueda Global",
command=self.abrir_búsqueda_global)
    search_menu.add_command(label="Búsqueda Avanzada",
command=self.abrir_búsqueda_avanzada)
    menubar.add_cascade(label="Buscar", menu=search_menu)

    # Menú Herramientas
    tools_menu = tk.Menu(menubar, tearoff=0)
    tools_menu.add_command(label="Gestor de Etiquetas",
command=self.abrir_gestor_etiquetas)
    tools_menu.add_command(label="Dashboard",
command=self.mostrar_dashboard)
    tools_menu.add_command(label="Asistente IA",
command=self.abrir_asistente_ia)
    menubar.add_cascade(label="Herramientas", menu=tools_menu)

    # Menú Reportes
    reports_menu = tk.Menu(menubar, tearoff=0)

```

```
reports_menu.add_command(label="Reporte de Actividad",
command=lambda: self.generar_reporte("actividad"))
reports_menu.add_command(label="Reporte Financiero",
command=lambda: self.generar_reporte("financiero"))
reports_menu.add_command(label="Reporte de Casos",
command=lambda: self.generar_reporte("casos"))
menubar.add_cascade(label="Reportes", menu=reports_menu)

# Menú Configuración
config_menu = tk.Menu(menubar, tearoff=0)
config_menu.add_command(label="Preferencias",
command=self.abrir_preferencias)
config_menu.add_command(label="Backup de Datos",
command=self.realizar_backup)
menubar.add_cascade(label="Configuración", menu=config_menu)

# Menú Ayuda
help_menu = tk.Menu(menubar, tearoff=0)
help_menu.add_command(label="Manual de Usuario",
command=self.abrir_manual)
help_menu.add_command(label="Acerca de",
command=self.mostrar_acerca_de)
menubar.add_cascade(label="Ayuda", menu=help_menu)

self.root.config(menu=menubar)
```

4. Timeline Detallado

4.1 Timeline por Fase

Fase	Duración	Semanas	Tareas Principales	Hitos
Fase 1: Consolidación	3 semanas	1-3	Merge branches, test suite, documentación	✓ Branches integradas ✓ Documentación completa
Fase 2: Funcionalidades Core	6 semanas	4-9	Etiquetas, Finanzas, Búsqueda	✓ Gestor Etiquetas ✓ Módulo Financiero ✓ Búsqueda Global
Fase 3: Mejoras UX	4 semanas	10-13	Navegación, Archivos, Personalización	✓ UI Optimizada ✓ Gestión Archivos Mejorada
Fase 4: Analytics	5 semanas	14-18	Dashboard, Reportes, Gráficos	✓ Dashboard Funcional ✓ Sistema Reportes
Fase 5: IA	8 semanas	19-26	Procesamiento Docs, Asistente, Generación	✓ Análisis Documentos ✓ Asistente Virtual
Fase 6: Escalabilidad	5 semanas	27-31	Multi-usuario, API, Cloud	✓ API REST ✓ Soporte Multi-usuario

4.2 Detalle Semanal - Fases Iniciales

Fase 1: Consolidación (Semanas 1-3)

Semana	Actividades	Entregables
Semana 1	• Merge feature/modulo-tareas • Pruebas iniciales • Corrección de conflictos	• Branch tareas integrada • Documentación de merge
Semana 2	• Merge feature/gestion-partes • Pruebas integradas • Configuración test suite	• Branch partes integrada • Estructura tests unitarios
Semana 3	• Documentación técnica • Refactorización • Corrección bugs identificados	• Documentación completa • Código refactorizado • Versión estable

Fase 2: Funcionalidades Core (Semanas 4-9)

Semana	Actividades	Entregables
Semana 4	• Diseño BD etiquetas • Implementación etiquetas backend	• Modelo de datos etiquetas • Funciones CRUD etiquetas
Semana 5	• UI gestión etiquetas • Integración con módulos existentes	• UI etiquetas funcional • Módulos core con soporte etiquetas
Semana 6	• Diseño BD finanzas • Backend presupuestos	• Modelo de datos finanzas • API presupuestos
Semana 7	• UI presupuestos • Backend pagos	• UI presupuestos • API pagos
Semana 8	• UI pagos • Reportes financieros	• UI pagos • Reportes básicos
Semana 9	• Implementación búsqueda FTS • UI búsqueda global	• Motor de búsqueda • Interfaz búsqueda

5. Recursos Necesarios

5.1 Equipo de Desarrollo

Rol	Habilidades	Dedicación	Fases
Desarrollador Principal	Python, tkinter, SQLite, MVC	100%	Todas
Desarrollador Secundario (opcional)	Python, UI/UX, Testing	50-100%	2-6
Especialista IA (consultor)	NLP, Machine Learning, LangChain	25%	5

5.2 Infraestructura y Herramientas

Herramienta	Uso	Notas
Git	Control de versiones	Preferiblemente GitHub
VS Code / PyCharm	Desarrollo	Con linting y formateo automático
SQLite Browser	Gestión BD	Para pruebas y desarrollo
PyInstaller	Compilación	Mantener compatibilidad
Pytest	Testing	Framework de testing unitario
Black	Formateo código	Mantener consistencia
Sphinx	Documentación	Generar docs técnicos

5.3 Dependencias de Software

Dependencias Base (existentes)

```
babel==2.17.0
pillow==11.2.1
plyer==2.1.0
pystray==0.19.5
six==1.17.0
tkcalendar==1.6.1
```

Nuevas Dependencias por Fase

```
# Fase 2-3
pyperclip==1.8.2
python-dateutil==2.8.2

# Fase 4 (Analytics)
matplotlib==3.7.1
reportlab==3.6.12
pandas==1.5.3
openpyxl==3.1.2

# Fase 5 (IA)
langchain==0.0.267
sentence-transformers==2.2.2
python-docx==0.8.11
pypdf2==3.0.1
pytesseract==0.3.10
```

6. Guías de Implementación

6.1 Estándares de Código

Estilo y Convenciones

- **Estilo:** PEP 8 para Python
- **Formateo:** Black con config estándar
- **Nomenclatura:**
 - snake_case para variables y funciones
 - CamelCase para clases
 - UPPER_CASE para constantes
- **Docstrings:** Google style para documentación

Ejemplo

```

def add_etiqueta(nombre, color="#3498db", descripcion=""):
    """Añade una nueva etiqueta al sistema.

    Args:
        nombre (str): Nombre único para la etiqueta
        color (str, optional): Color en formato hex. Default:
"#3498db"
        descripcion (str, optional): Descripción opcional

    Returns:
        int: ID de la etiqueta creada o None si hubo error

    Raises:
        sqlite3.IntegrityError: Si ya existe una etiqueta con ese
nombre
    """
    conn = connect_db()
    if conn:
        try:
            cursor = conn.cursor()
            timestamp = int(time.time())
            cursor.execute(
                """
                INSERT INTO etiquetas (nombre, color, descripcion,
created_at)
                VALUES (?, ?, ?, ?)
                """,
                (nombre, color, descripcion, timestamp)
            )
            conn.commit()
            return cursor.lastrowid
        except sqlite3.Error as e:
            print(f"Error al crear etiqueta: {e}")
            conn.rollback()
            return None

```

```
finally:  
    close_db(conn)
```

6.2 Patrones a Seguir

Patrones Establecidos

- **MVC Modificado:** Mantener separación Model-View-Controller
- **Observer Pattern:** Para comunicación entre componentes
- **State Pattern:** Para gestión de estados de UI
- **Template Method:** Para operaciones CRUD
- **Factory Pattern:** Para creación de conexiones y objetos

Patrones Adicionales Recomendados

- **Composite Pattern:** Para componentes UI anidables
- **Strategy Pattern:** Para diferentes implementaciones de reportes
- **Command Pattern:** Para operaciones undoables (donde aplique)
- **Repository Pattern:** Para abstraer acceso a datos

6.3 Guía de UI/UX

Principios de Diseño

- **Consistencia:** Mantener patrones de UI consistentes
- **Feedback:** Indicadores claros de acciones y estados
- **Eficiencia:** Atajos y accesos rápidos para usuarios avanzados
- **Jerarquía:** Organización clara de información por importancia
- **Accesibilidad:** Contraste adecuado y tamaños legibles

Componentes UI Estándar

```

class StandardDialog(tk.Toplevel):
    """Diálogo estándar con estructura común."""

    def __init__(self, parent, title, width=400, height=300):
        super().__init__(parent)
        self.title(title)
        self.transient(parent)
        self.grab_set()
        self.resizable(False, False)

        # Centrar en pantalla
        x = parent.winfo_rootx() + (parent.winfo_width() -
width) // 2
        y = parent.winfo_rooty() + (parent.winfo_height() -
height) // 2
        self.geometry(f"{width}x{height}+{x}+{y}")

        # Frame principal con padding
        self.main_frame = ttk.Frame(self, padding="15")
        self.main_frame.pack(fill=tk.BOTH, expand=True)

        # Botones estándar
        self.button_frame = ttk.Frame(self.main_frame)
        self.button_frame.pack(side=tk.BOTTOM, fill=tk.X, pady=(15,
0))

        self.cancel_btn = ttk.Button(
            self.button_frame, text="Cancelar",
command=self.destroy
        )
        self.cancel_btn.pack(side=tk.RIGHT, padx=5)

        self.ok_btn = ttk.Button(
            self.button_frame, text="Aceptar",
command=self.on_accept

```

```
)  
self.ok_btn.pack(side=tk.RIGHT, padx=5)  
  
def on_accept(self):  
    """Override para implementar acción de aceptar."""  
    pass
```

6.4 Estrategia de Integración

Puntos de Extensión

1. **Sistema de Pestañas:** Usar `main_notebook.add()` para nuevos módulos
2. **Menú Principal:** Añadir items al menubar existente
3. **Event Bus:** Suscribirse a eventos relevantes para reaccionar a cambios
4. **Tooltips y Ayuda:** Contextual en cada componente nuevo

Proceso de Integración de Nuevos Módulos

1. Crear clase derivada de `ttk.Frame` en archivo separado (ej: `etiquetas_ui.py`)
 2. Implementar `__init__` con referencia a `app_controller`
 3. Implementar UI interna y lógica de componente
 4. Registrar en `ModuleRegistry` de `main_app.py`
 5. Crear handlers para eventos relevantes
 6. Añadir funciones de BD necesarias en `crm_database.py`
-

7. Plan de Testing

7.1 Estrategia de Testing

Tipos de Pruebas

- **Unitarias:** Funciones individuales
- **Integración:** Interacción entre componentes
- **UI:** Validación de interfaces
- **Sistema:** Flujos completos
- **Regresión:** Verificar funcionalidad existente

Herramientas

- **pytest:** Framework principal de testing
- **pytest-mock:** Para mock objects
- **pytest-cov:** Cobertura de código
- **pytest-tk:** Para testing de interfaces tkinter

7.2 Estructura de Tests

```
tests/
├─ unit/
│   ├─ test_database.py
│   ├─ test_etiquetas.py
│   ├─ test_finanzas.py
│   └─ ...
├─ integration/
│   ├─ test_etiquetas_ui.py
│   ├─ test_busqueda_global.py
│   └─ ...
├─ ui/
│   ├─ test_dialogs.py
│   ├─ test_treeviews.py
│   └─ ...
└─ system/
    ├─ test_flujo_cliente_caso.py
    ├─ test_flujo_financiero.py
    └─ ...
```

7.3 Casos de Prueba por Módulo

Gestor de Etiquetas

- **Unitarias:**
 - Crear/modificar/eliminar etiquetas
 - Asignar/desasignar etiquetas a entidades
 - Buscar entidades por etiqueta
- **Integración:**
 - Visualización correcta en vistas de lista
 - Filtrado por etiquetas

- **UI:**
- Diálogo de gestión de etiquetas
- Selector de etiquetas
- Color picker

Módulo Financiero

- **Unitarias:**
- CRUD de presupuestos e ítems
- Cálculos de totales y subtotales
- Registro y cálculos de pagos
- **Integración:**
- Vinculación con casos
- Actualización de estados
- **UI:**
- Formulario de presupuesto
- Lista de pagos
- Reportes financieros

7.4 Automatización y CI/CD

Configuración de pytest

```
# conftest.py
import pytest
import tkinter as tk
import os
import sqlite3
import tempfile

@pytest.fixture
def test_db():
    """Crea una BD temporal para testing."""
    db_fd, db_path = tempfile.mkstemp()
    conn = sqlite3.connect(db_path)

    # Crear esquema
    with open('schema.sql', 'r') as f:
        conn.executescript(f.read())

    yield db_path

    # Cleanup
    os.close(db_fd)
    os.unlink(db_path)

@pytest.fixture
def app_root():
    """Crea una ventana raíz para testing UI."""
    root = tk.Tk()
    root.withdraw() # Ocultar ventana en tests
    yield root
    root.destroy()
```

Ejemplos de Tests Unitarios

```
# test_database.py
def test_add_etiqueta(test_db):
    # Setup
    import crm_database as db
    db.DATABASE_FILE = test_db

    # Test
    etiqueta_id = db.add_etiqueta("Test", "#FF0000", "Etiqueta de prueba")
    assert etiqueta_id is not None

    # Verificación
    etiquetas = db.get_etiquetas()
    assert len(etiquetas) == 1
    assert etiquetas[0]['nombre'] == "Test"
    assert etiquetas[0]['color'] == "#FF0000"
```

8. Consideraciones de Mantenimiento

8.1 Estrategia de Versionado

Esquema de Versiones

- **MAJOR.MINOR.PATCH** (e.g., 1.0.0)
- **MAJOR**: Cambios incompatibles de API
- **MINOR**: Nuevas funcionalidades compatibles
- **PATCH**: Corrección de bugs

Notas de Versión

```
# Ejemplo de CHANGELOG.md

## [1.0.0] - 2025-12-15
### Añadido
- Sistema completo de etiquetas
- Módulo financiero con presupuestos y pagos
- Búsqueda global con FTS

### Mejorado
- Rendimiento en carga de casos
- UI más responsiva
- Navegación con keyboard shortcuts

### Corregido
- Problema de memoria con muchos casos
- Error en exportación de calendario
- Visualización incorrecta en pantallas pequeñas
```

8.2 Gestión de Dependencias

- Mantener `requirements.txt` actualizado
- Pinning de versiones específicas
- Documentar dependencias por módulo
- Revisar compatibilidad con PyInstaller

8.3 Migración de Base de Datos

Sistema de Versiones de Esquema

```
def get_schema_version():
    """Obtiene versión actual del esquema de BD."""
    conn = connect_db()
    if conn:
        try:
            cursor = conn.cursor()
            cursor.execute("PRAGMA user_version")
            version = cursor.fetchone()[0]
            return version
        finally:
            close_db(conn)
    return 0

def update_schema():
    """Actualiza esquema de BD según versión actual."""
    current_version = get_schema_version()

    if current_version < 1:
        # Migración a v1
        execute_migration("migrations/v1_add_etiquetas.sql")
        set_schema_version(1)

    if current_version < 2:
        # Migración a v2
        execute_migration("migrations/v2_add_finanzas.sql")
        set_schema_version(2)

    # Continuar con más migraciones...
```

8.4 Respaldo y Recuperación

Estrategia de Backup

```
def crear_backup():
    """Crea backup de la BD actual."""
    fecha = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    backup_path = f"backups/crm_legal_backup_{fecha}.db"

    # Asegurar que existe directorio
    os.makedirs(os.path.dirname(backup_path), exist_ok=True)

    conn = connect_db()
    if conn:
        try:
            # Backup con conexión abierta
            backup = sqlite3.connect(backup_path)
            conn.backup(backup)
            backup.close()
            return backup_path
        except sqlite3.Error as e:
            print(f"Error al crear backup: {e}")
            return None
    finally:
        close_db(conn)
```

8.5 Monitoreo de Rendimiento

```

class PerformanceMonitor:
    def __init__(self):
        self.operations = {}

    def start_operation(self, name):
        """Inicia tracking de una operación."""
        return (name, time.time())

    def end_operation(self, operation_data):
        """Finaliza tracking y registra tiempo."""
        name, start_time = operation_data
        duration = time.time() - start_time

        if name not in self.operations:
            self.operations[name] = []

        self.operations[name].append(duration)

        # Log si es muy lento
        if duration > 1.0: # Más de 1 segundo
            print(f"Operación lenta: {name} tomó {duration:.2f} segundos")

    def get_avg_duration(self, name):
        """Obtiene duración promedio de una operación."""
        if name in self.operations and self.operations[name]:
            return sum(self.operations[name]) /
len(self.operations[name])
        return 0

    def get_stats(self):
        """Obtiene estadísticas de todas las operaciones."""
        stats = {}
        for name, durations in self.operations.items():
            if durations:

```

```
        stats[name] = {
            'count': len(durations),
            'avg': sum(durations) / len(durations),
            'min': min(durations),
            'max': max(durations)
        }
    return stats
```

9. Resumen por Fase

Fase 1: Consolidación y Fortalecimiento

- **Objetivos:**
 - Integrar branches desarrolladas
 - Establecer infraestructura de testing
 - Documentar el sistema
 - Refactorizar áreas críticas
- **Entregables:**
 - Sistema consolidado y estable
 - Framework de testing unitario
 - Documentación técnica completa
- **Duración:** 3 semanas
- **Dependencias:** Ninguna

Fase 2: Funcionalidades Core

- **Objetivos:**
 - Implementar sistema transversal de etiquetas
 - Crear módulo completo de finanzas

- Implementar búsqueda global
- **Entregables:**
- Gestor de etiquetas funcional
- Módulo de presupuestos y pagos
- Sistema de búsqueda FTS
- **Duración:** 6 semanas
- **Dependencias:** Fase 1

Fase 3: Mejoras de Usabilidad

- **Objetivos:**
- Optimizar experiencia de usuario
- Mejorar navegación y accesibilidad
- Implementar funciones avanzadas de archivos
- **Entregables:**
- Atajos de teclado y navegación mejorada
- Gestión avanzada de documentos
- Personalización de interfaz
- **Duración:** 4 semanas
- **Dependencias:** Fase 2

Fase 4: Analytics y Reportes

- **Objetivos:**
- Crear dashboard con KPIs
- Implementar sistema de reportes
- Desarrollar visualizaciones gráficas
- **Entregables:**
- Dashboard funcional

- Reportes exportables
- Sistema de gráficos
- **Duración:** 5 semanas
- **Dependencias:** Fase 2

Fase 5: Integración de IA

- **Objetivos:**
 - Implementar análisis automático de documentos
 - Desarrollar asistente virtual
 - Crear generación de contenido
- **Entregables:**
 - OCR y análisis de documentos
 - Chatbot asistente
 - Templates inteligentes
- **Duración:** 8 semanas
- **Dependencias:** Fase 3

Fase 6: Escalabilidad y Proyección

- **Objetivos:**
 - Preparar sistema para multi-usuario
 - Implementar API REST
 - Habilitar integración con cloud
- **Entregables:**
 - Arquitectura multi-usuario
 - API documentada
 - Funciones de sincronización
- **Duración:** 5 semanas

Conclusiones

El CRM Legal presenta una base técnica sólida con arquitectura modular y extensible que facilita la implementación de las funcionalidades pendientes. El plan de desarrollo propuesto prioriza la consolidación de las características existentes, seguido por la implementación incremental de módulos de alto valor para los usuarios.

Con un enfoque en calidad de código, patrones consistentes, y testing riguroso, este plan garantiza un desarrollo mantenible y robusto. El timeline propuesto de 31-37 semanas es realista considerando la complejidad de las funcionalidades, aunque puede ajustarse según la disponibilidad de recursos.

La implementación secuencial de fases permite entregas de valor incremental, dando prioridad a funcionalidades con mayor impacto en usuarios (etiquetas, finanzas, búsqueda) antes de avanzar hacia capacidades más avanzadas (analytics, IA).

Recomendación final: Proceder con Fase 1 inmediatamente para consolidar avances existentes, mientras se planifica en detalle la implementación de la Fase 2, que aportará valor crítico a los usuarios del sistema.

Plan desarrollado: Junio 2025